

Data Structure(CSL202)

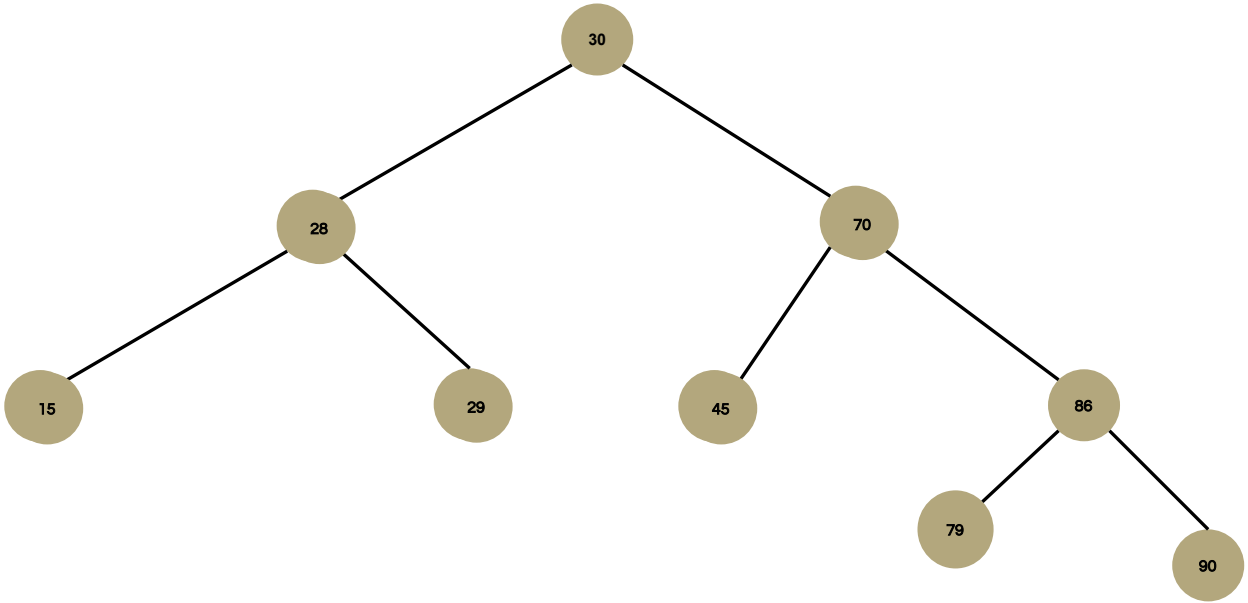
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)



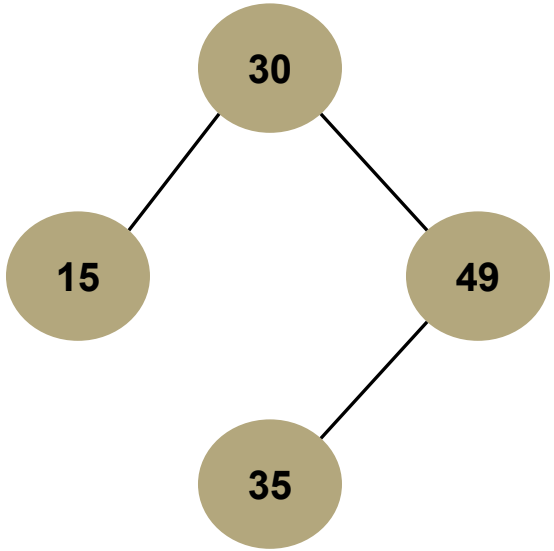
Binary Search Tree(BST)

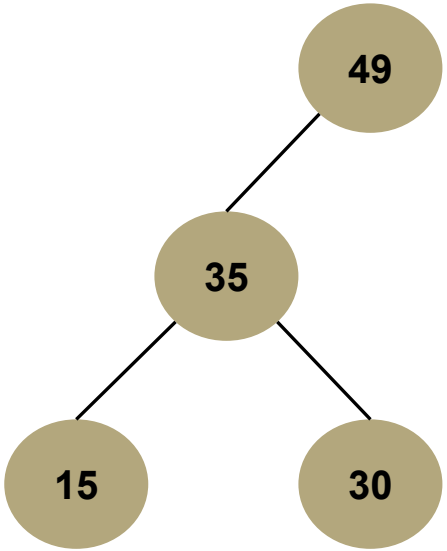
Binary Search Tree(BST)

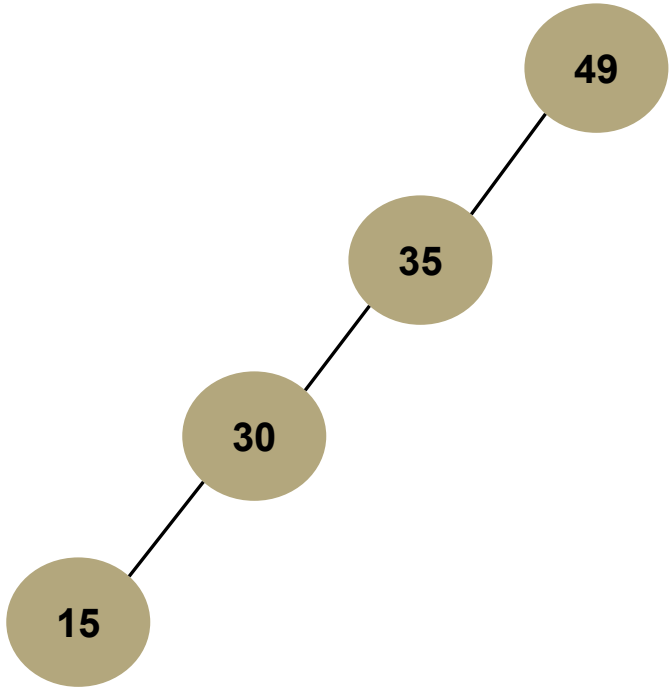
- It is a binary tree where each node x has a key, key(x).
- Any node y in the left subtree of x have key(y) $\leq$ key(x).
- Any node y in the right subtree of x have key(y) $\geq$ key(x).



Same set of keys may result in different BSTs.

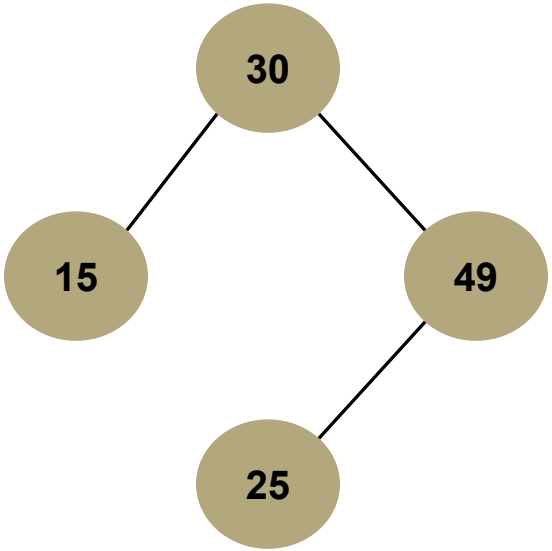




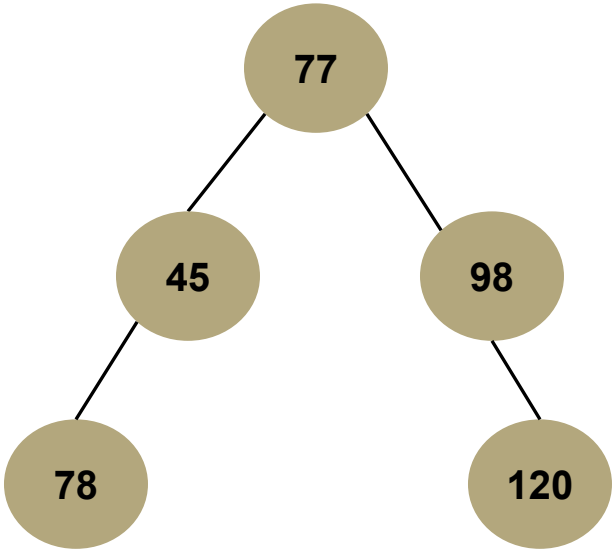


Identify BST

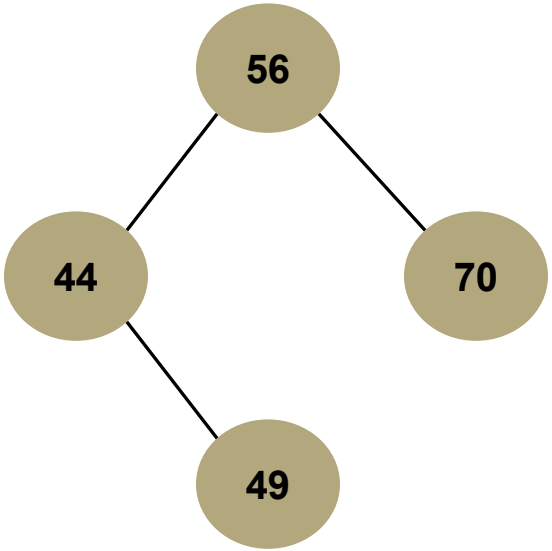
Which of the following are BSTs?



(a)



(b)



(c)

Operations in BST



Searching in BST

```
search(BST T, int k)
1. n := root(T)                    // Start at the root of the tree
2. while n ≠ NULL do               // Continue until the node becomes NULL
3.     if key(n) = k then          // If current node matches the key
4.         break                   // Found the node, exit the loop
5.     if key(n) > k then          // If key is smaller, go to left subtree
6.         n := left(n)
7.     else                        // Else, go to right subtree
8.         n := right(n)
9. return n                        // Return the node (or NULL if not found)
```

Searching in BST (C Code)

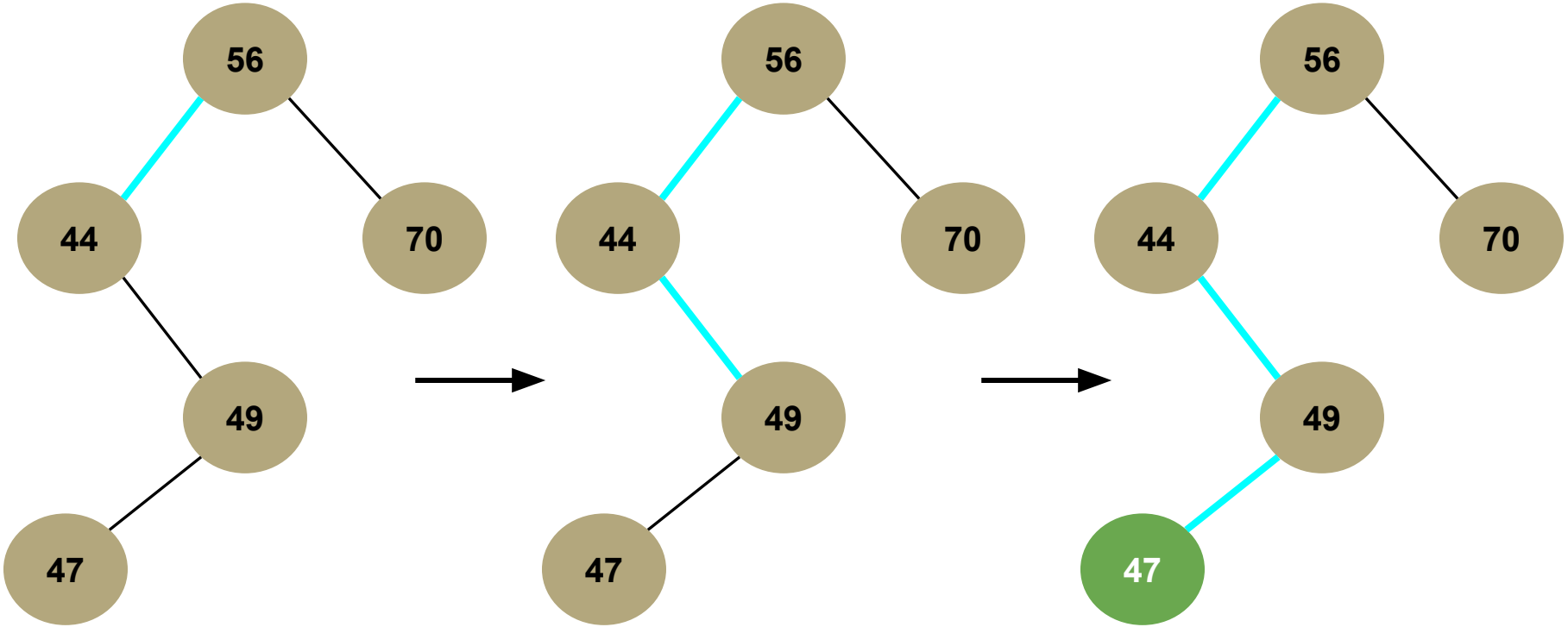
```
struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)
        return search(root->left, key);

    return search(root->right, key);
}
```

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

Searching in BST

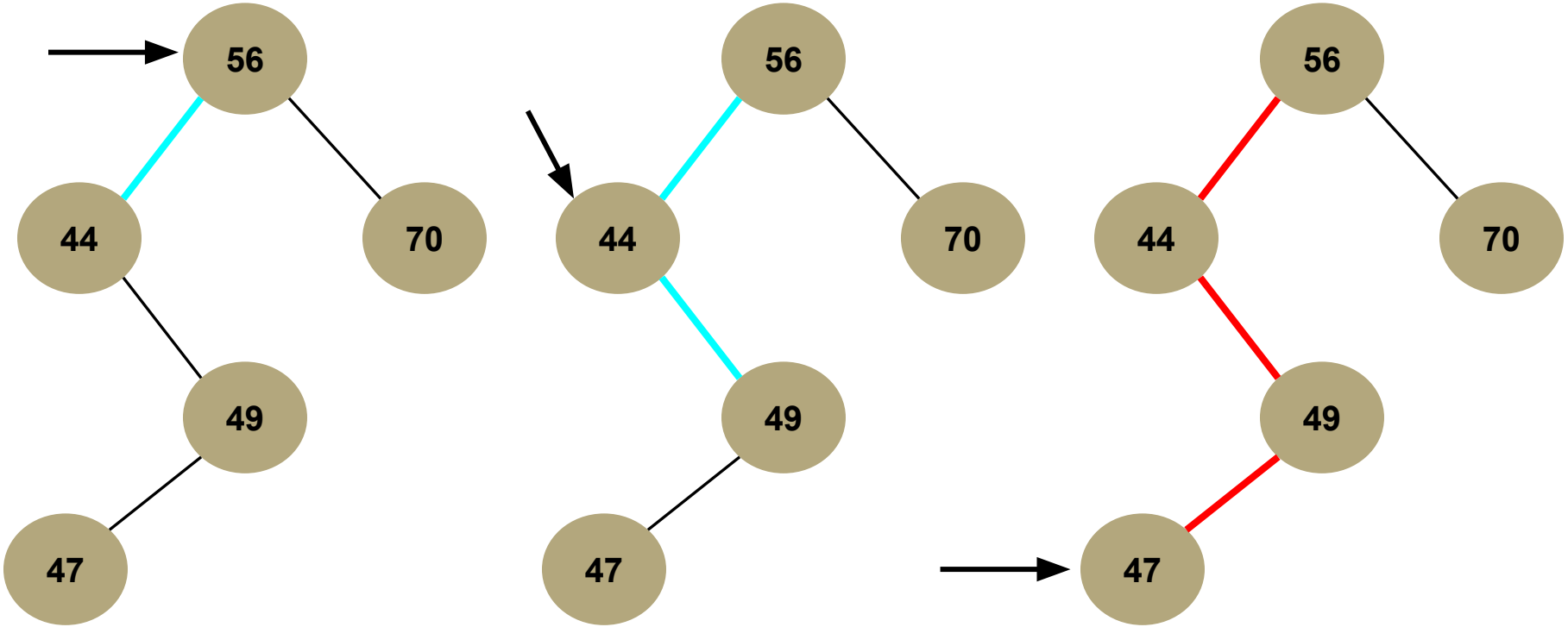
Search for 47



Search successful!

Searching in BST

Search for 45



Search unsuccessful!

Searching in BST Complexity

Best/Average Case:

- If the BST is balanced, height h ≈ log n.
- So, running time = O(log n).

Worst Case:

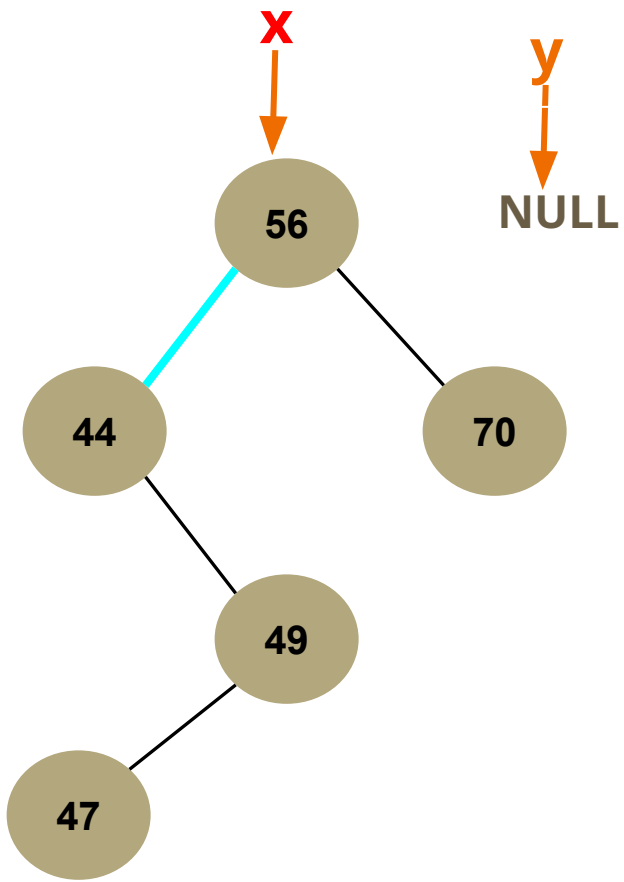
- If the BST is skewed (like a linked list), height h = n.
- So, running time = O(n).

Operations in BST



Insertion in BST

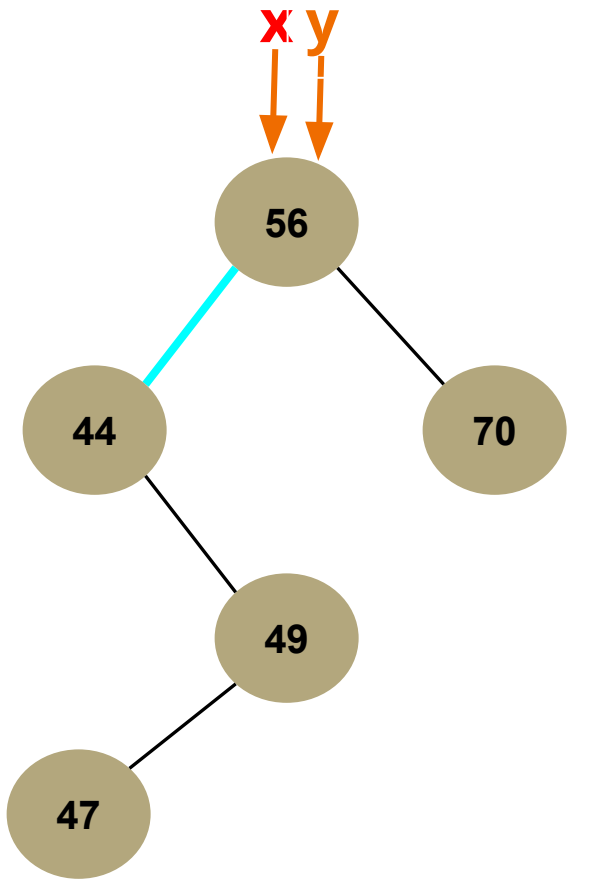
```
Insert(BST T, int n)
// Start from the root. y tracks the parent.
1. x := root(T); y := NULL
// Traverse the tree to find insertion point
2. while x ≠ NULL do
3.     y := x
4.     if key(n) < key(x) then
           // Move to left subtree
5.         x := left(x)
6.     Else
           // Move to right subtree
7.         x := right(x)
```



key(n)=45

Insertion in BST

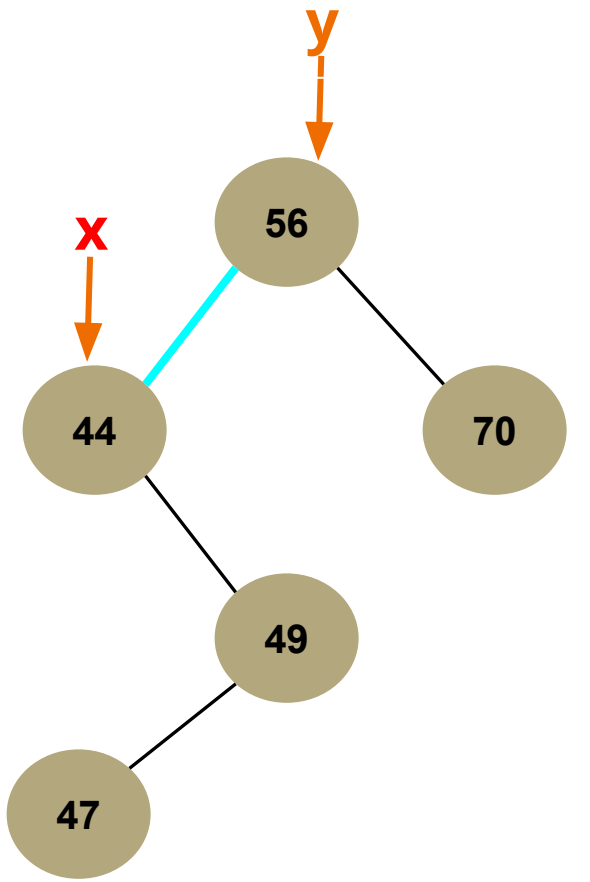
```
Insert(BST T, int n)
// Start from the root. y tracks the parent.
1. x := root(T); y := NULL
// Traverse the tree to find insertion point
2. while x ≠ NULL do
3.     y := x
4.     if key(n) < key(x) then
           // Move to left subtree
5.         x := left(x)
6.     Else
           // Move to right subtree
7.         x := right(x)
```



key(n)=45

Insertion in BST

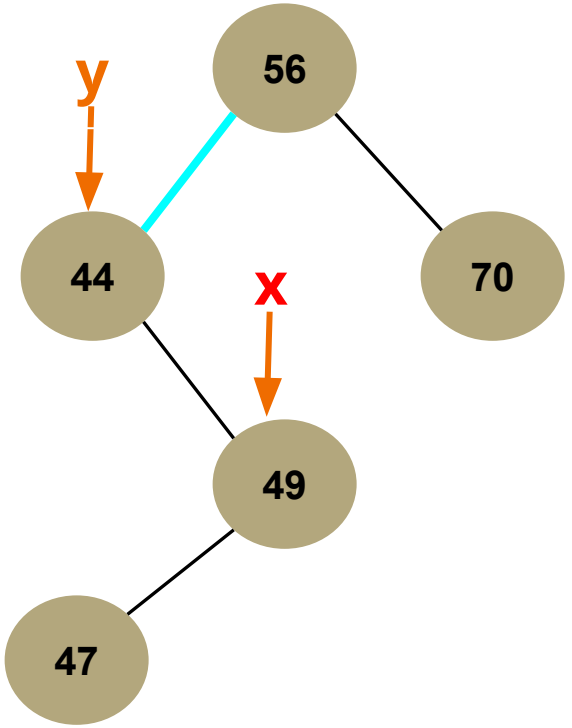
```
Insert(BST T, int n)
// Start from the root. y tracks the parent.
1. x := root(T); y := NULL
// Traverse the tree to find insertion point
2. while x ≠ NULL do
3.     y := x
4.     if key(n) < key(x) then
           // Move to left subtree
5.         x := left(x)
6.     Else
           // Move to right subtree
7.         x := right(x)
```



key(n)=45

Insertion in BST

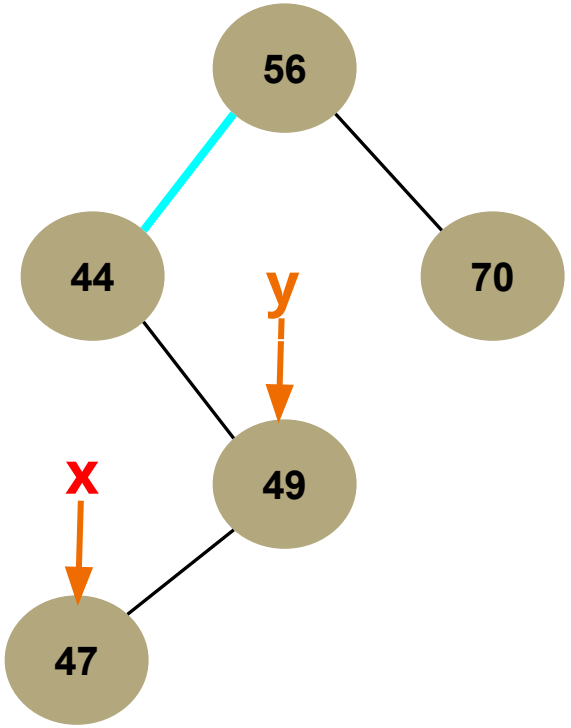
```
Insert(BST T, int n)
// Start from the root. y tracks the parent.
1. x := root(T); y := NULL
// Traverse the tree to find insertion point
2. while x ≠ NULL do
3.     y := x
4.     if key(n) < key(x) then
           // Move to left subtree
5.         x := left(x)
6.     Else    // Move to right subtree
7.         x := right(x)
```



key(n)=45

Insertion in BST

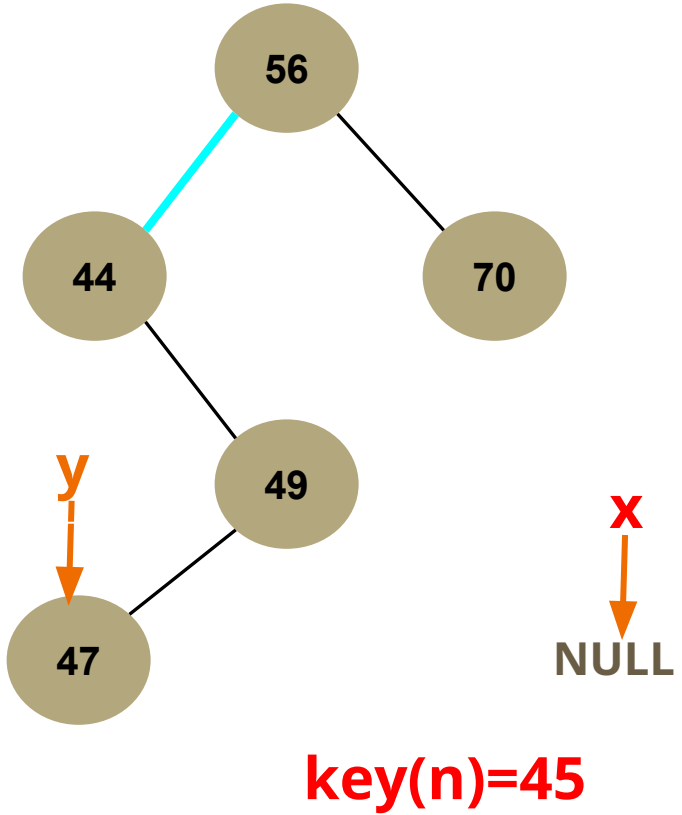
```
Insert(BST T, int n)
// Start from the root. y tracks the parent.
1. x := root(T); y := NULL
// Traverse the tree to find insertion point
2. while x ≠ NULL do
3.     y := x
4.     if key(n) < key(x) then
           // Move to left subtree
5.         x := left(x)
6.     Else            // Move to right subtree
7.         x := right(x)
```



key(n)=45

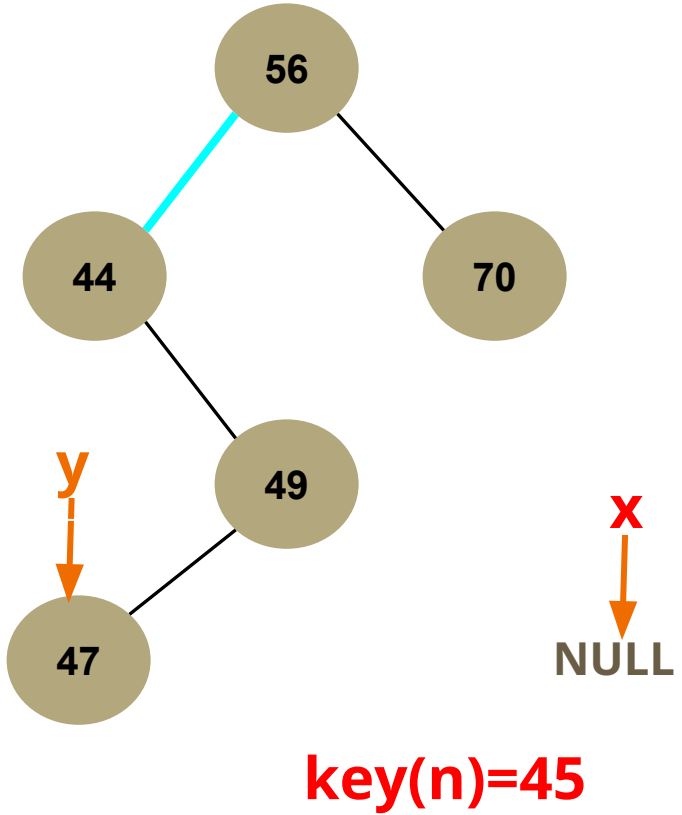
Insertion in BST

```
Insert(BST T, int n)
// Start from the root. y tracks the parent.
1. x := root(T); y := NULL
// Traverse the tree to find insertion point
2. while x ≠ NULL do
3.     y := x
4.     if key(n) < key(x) then
           // Move to left subtree
5.         x := left(x)
6.     Else            // Move to right subtree
7.         x := right(x)
```



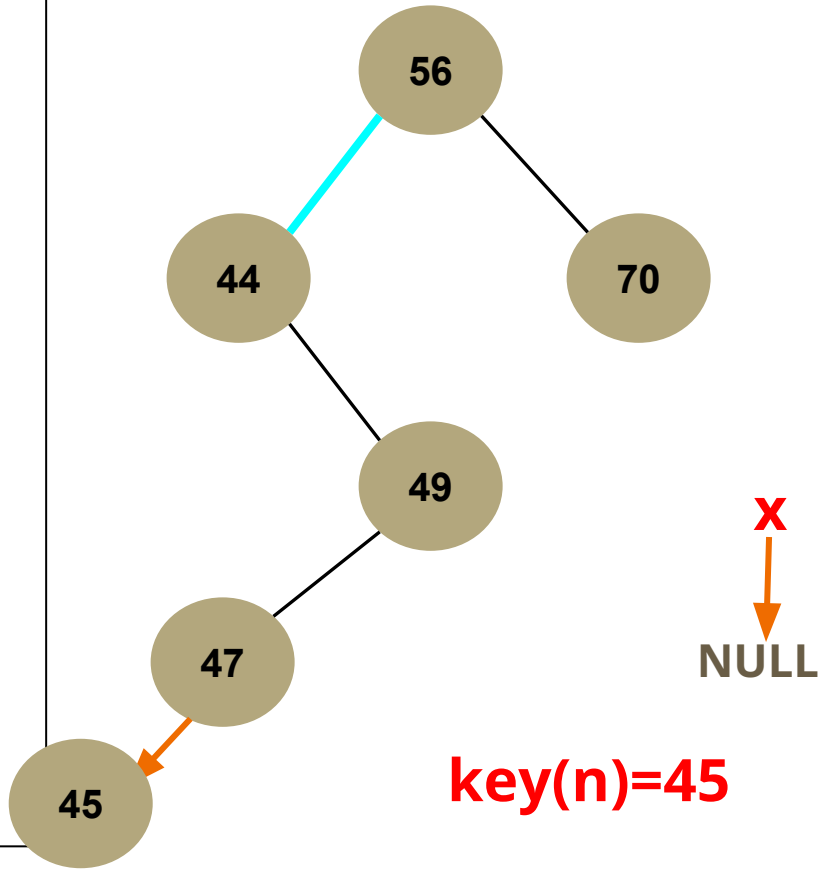
Insertion in BST

```
// Tree was empty
8. if y = NULL then
9.     root(T) := n
10. else if key(n) < key(y) then
        // Attach n to the left of y
11.     left(y) := n
12. Else
        // Attach n to the right of y
13.     right(y) := n
       // Set parent of n
14. parent(n) := y
```



Insertion in BST

```
// Tree was empty
8. if y = NULL then
9.     root(T) := n
10. else if key(n) < key(y) then
        // Attach n to the left of y
11.     left(y) := n
12. Else
        // Attach n to the right of y
13.     right(y) := n
       // Set parent of n
14. parent(n) := y
```



Insertion in BST

```
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);  // First node
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    // If value already exists, do nothing
    return root;
}
```

Insertion in BST Complexity

Best/Average Case:

- If the BST is balanced, height $h \approx \log n$.
- So, running time = $O(\log n)$.

Worst Case:

- If the BST is skewed (like a linked list), height $h = n$.
- So, running time = $O(n)$.